HARBOR: Risk-Aware Portfolio Framework — Technical PRD v1.0

Project: HARBOR (Hierarchical Autonomous Risk-Based Optimization & Research)
Type: Open-Source Python Framework
Version: 1.0.0
Date: February 18, 2026
Owner: Lewis Smith

Executive Summary

HARBOR is a research-grade, open-source portfolio optimization and risk management framework built for depth over breadth. It implements state-of-the-art mathematical models (Hierarchical Risk Parity, Monte Carlo simulation, risk decomposition) and modern ML/RL/DL techniques in a clean, modular architecture designed for autonomous operation and scientific reproducibility.

Core philosophy: Few components, deep implementation, rigorous math, production-quality code.

Objectives

- Provide a mathematically rigorous portfolio construction and risk management toolkit that institutional researchers can audit, extend, and trust
- Implement advanced techniques (HRP, Monte Carlo VaR/CVaR, neural volatility forecasting, deep RL agents) with full mathematical transparency and reproducibility
- Serve as the computational foundation for Artificial Behavioral Finance (ABF) research—enabling experiments on systematic strategy behavior, crowding, and correlation regimes

- Maintain production-grade engineering standards (tests, CI, versioning, docs) while prioritizing research flexibility

Non-Goals

- Live trading or execution infrastructure (research/backtest only)
- Supporting every asset class or exotic derivative (focus: equities, with clean extensibility)
- Building a GUI or web dashboard (command-line and notebook interfaces only)
- Competing with commercial platforms (this is open research infrastructure)

Technical Architecture

Core Modules

harbor/
data/
loaders.py # SP500, CRSP, survivorship-bias-free loaders
universe.py # Historical constituent membership
risk/
hierarchical.py # Hierarchical Risk Parity (HRP)
monte_carlo.py # Monte Carlo VaR/CVaR simulation
decomposition.py # Risk attribution and marginal risk
correlation.py # Dynamic correlation estimation, regime detection
portfolio/
optimization.py # Mean-variance, risk-parity, constrained optimization
rebalancing.py # Turnover-aware rebalancing logic
signals/
momentum.py # Time-series and cross-sectional momentum
volatility.py # Realized vol, GARCH, vol-targeting signals
crowding.py # Signal similarity and crowding proxies
ml/
volatility/
nn_forecaster.py # LSTM/GRU neural volatility models
training.py # Training loops, validation, hyperparameter search
behavior_agents/
rl_agent.py # Deep RL portfolio agents (PPO-style)

envs.py # Gym-compatible portfolio environments
rewards.py # Behavioral reward functions (loss aversion, etc.)
backtest/
engine.py # Event-driven backtester
metrics.py # Sharpe, Sortino, max DD, Calmar, VaR metrics
attribution.py # Performance attribution by signal/factor
abf/
q1_persistence.py # ABF Question 1 pipeline
q2_crowding.py # ABF Question 2 pipeline

Design Principles

1. **Mathematical transparency:** Every risk model includes docstring with closed-form equations and citations
2. **Reproducibility:** Fixed random seeds, versioned configs, deterministic pipelines
3. **Modularity:** Each component is independently testable and replaceable
4. **Performance:** Vectorized operations (NumPy/Pandas), optional GPU for DL (PyTorch)

Deep Technical Components

1. Hierarchical Risk Parity (HRP)

Mathematical foundation:

HRP addresses instability in mean-variance optimization by using hierarchical clustering and recursive bisection to allocate risk[1].

Algorithm:

1. Compute correlation matrix C from returns
2. Use hierarchical clustering (e.g., single-linkage) on distance metric $d_{ij} = \sqrt{(1 - \rho_{ij})/2}$
3. Perform recursive bisection to allocate risk inversely proportional to cluster variance

Implementation requirements:

- Covariance estimation: sample covariance, shrinkage estimators (Ledoit-Wolf), or exponentially weighted
- Distance metrics: Euclidean on correlations, or alternative linkages (Ward, complete)
- Validation: compare against naive risk parity and mean-variance

Output: Weight vector $w \in \mathbb{R}^N$ satisfying $\sum w_i = 1, w_i \geq 0$, optimized for diversification without relying on expected returns.

2. Monte Carlo Risk Simulation

Purpose: Estimate tail risk (VaR, CVaR/ES) and stress-test portfolios under simulated scenarios.

Methods:

- **Historical simulation:** Bootstrap from historical return distribution
- **Parametric simulation:** Assume multivariate normal or Student-t, draw N scenarios
- **GARCH-based simulation:** Forecast volatility paths, simulate returns conditional on vol dynamics

Metrics computed:

$$\mathrm{VaR}_\alpha = -\inf\{x : P(L \leq x) \geq \alpha\}$$

$$\mathrm{CVaR}_\alpha = \mathbb{E}[L \mid L \geq \mathrm{VaR}_\alpha]$$

where L is portfolio loss, α is confidence level (e.g., 0.95, 0.99).

Implementation:

- 10,000+ simulated paths per risk calculation
- Antithetic variates for variance reduction
- Parallel simulation for large universes (multiprocessing or GPU)

3. Neural Volatility Forecasting

Motivation: Improve volatility estimates for risk models and vol-targeting strategies using recurrent neural networks[2][3].

Architecture:

- **Input:** Rolling window of returns, realized volatility, volume (e.g., 60-day lookback)
- **Model:** LSTM or GRU with 2–3 hidden layers, dropout for regularization
- **Output:** Forecasted volatility $\hat{\sigma}_{t+h}$ for horizons $h \in \{1, 5, 21\}$ days

Training:

- Loss: MSE on volatility forecast error, or quantile loss for distributional forecasts
- Validation: rolling-window cross-validation to avoid lookahead bias
- Benchmarks: compare against historical vol, GARCH(1,1), realized vol

Risk integration:

Use $\hat{\sigma}_{t+1}$ from neural forecaster in:

- Volatility-targeting rules: scale position size inversely with $\hat{\sigma}$
- Covariance matrix estimation: replace diagonal elements with neural forecasts

4. Deep RL Behavioral Agents

Purpose: Model portfolio agents with behavioral biases (loss aversion, overconfidence) using deep reinforcement learning[4][5].

Environment:

- State: portfolio weights, returns, volatility, crowding proxies
- Actions: weight adjustments (continuous or discretized)
- Reward: modified Sharpe ratio with behavioral penalties (e.g., extra penalty for drawdowns to encode loss aversion)

Agent architecture:

- Actor: neural network mapping state s_t to action a_t (policy $\pi_\theta(a|s)$)
- Critic: neural network estimating value function $V_\phi(s)$ or $Q_\phi(s, a)$
- Algorithm: PPO (Proximal Policy Optimization) or SAC (Soft Actor-Critic)

Behavioral modifications:

- **Loss aversion:** Asymmetric reward: $r_t = \begin{cases} \alpha \cdot \text{gain} & \text{if gain} \\ \beta \cdot \text{loss} & \text{if loss} \end{cases}$, $\beta > \alpha$
- **Overconfidence:** Underweight volatility in state representation or reward
- **Disposition effect:** Bias toward holding losers, selling winners (encoded in action penalties)

Research application:

Train multiple agents with similar features but different behavioral parameters, then simulate their collective behavior to study crowding and correlation amplification (ABF Question 2).

5. Risk Decomposition and Attribution

Marginal risk contribution:

For asset i in portfolio with weights w, covariance Σ:

$$\text{MRC}_i = \frac{\partial \sigma_p}{\partial w_i} = \frac{(\Sigma w)_i}{\sigma_p}$$

Risk contribution:

$$\text{RC}_i = w_i \cdot \text{MRC}_i$$

Validation: $\sum_{i=1}^{N} \text{RC}_i = \sigma_p$ (portfolio volatility).

Use cases:

- Identify concentrated risk exposures
- Rebalance toward equal risk contribution (risk parity)
- Detect when single assets dominate portfolio risk

ML/RL Technical Depth

Volatility Forecasting — Detailed Spec

Data preprocessing:

- Returns: log returns $r_t = \log(P_t/P_{t-1})$
- Realized volatility: $\mathrm{RV}_t = \sqrt{\sum_{i=1}^{n} r_{t,i}^2}$ (intraday if available, else close-to-close)
- Normalization: z-score standardization per asset

Model:

```
LSTM(
input_size=3, # [return, realized_vol, volume]
hidden_size=64,
num_layers=2,
dropout=0.2
)
Linear(64, 1) # Output: volatility forecast
```

Training procedure:

1. Split: 70% train, 15% validation, 15% test (chronological, no shuffling)
2. Optimizer: Adam, learning rate 0.001 with ReduceLROnPlateau
3. Early stopping: patience=10 epochs on validation loss
4. Batch size: 32, epochs: 100 max

Evaluation metrics:

- Statistical: MSE, MAE on volatility forecast
- Risk: VaR violation rate (proportion of days where realized loss exceeds predicted VaR)
- Economic: Sharpe ratio of vol-targeted portfolio using NN forecasts vs GARCH

Deep RL Agent — Detailed Spec

Environment (Gym-compatible):

class PortfolioEnv:
observation_space: Box(shape=(N_assets * 3,)) # [weights, returns, vols]
action_space: Box(shape=(N_assets,), low=-0.1, high=0.1) # weight changes

```
def step(action):
    # Apply action, compute new weights, calculate reward
    # Deduct transaction costs proportional to turnover
    return next_state, reward, done, info
```

Agent (PPO):

Actor: MLP(state_dim, [256, 128], action_dim) with tanh activation
Critic: MLP(state_dim, [256, 128], 1) with linear output
Policy: Gaussian with learned std
Training: PPO clip=0.2, GAE lambda=0.95, 2048 steps per update

Behavioral reward (loss aversion example):

def reward(returns, weights, prev_weights):
pnl = np.dot(returns, weights)
turnover_cost = transaction_cost * np.sum(np.abs(weights - prev_weights))

```
if pnl >= 0:
    behavioral_pnl = pnl  # Normal gain
else:
    behavioral_pnl = 2.5 * pnl  # Amplify loss (loss aversion)

return behavioral_pnl - turnover_cost
```

Training:

- Train for 500k steps over historical SP500 data (daily rebalancing)
- Save checkpoints every 50k steps
- Evaluate on held-out test period (2024–2026)

Integration with ABF Research

HARBOR provides the computational substrate for ABF experiments:

ABF Component	HARBOR Module
Vol-targeting strategies	signals/volatility.py, portfolio/optimization.py
Shock detection	risk/correlation.py, abf/q1_persistence.py
Crowding proxies	signals/crowding.py, abf/q2_crowding.py
Agent simulations	ml/behavior_agents/
Backtest validation	backtest/engine.py, backtest/metrics.py

Research findings from ABF (e.g., "reduce exposure when crowding proxy > threshold") are implemented as regime-aware portfolio construction rules and validated via backtest comparison.

Success Metrics

Engineering Quality

- **Test coverage:** $\geq 80\%$ for core modules (risk, portfolio, backtest)
- **Documentation:** Every public function has docstring with equations and references
- **CI/CD:** GitHub Actions running tests on every PR
- **Reproducibility:** All experiments runnable with single `make reproduce` command

Technical Depth

- HRP implementation validated against published benchmarks (e.g., Lopez de Prado papers)
- Monte Carlo VaR backtested against realized losses (violation rate within $\pm 2\%$ of nominal)

- Neural volatility forecaster beats GARCH baseline by $\geq 10\%$ on MSE and VaR violation rate
- Deep RL agent achieves Sharpe ratio ≥ 1.0 on test period with realistic transaction costs

Research Enablement

- ABF Q1 and Q2 experiments run end-to-end using HARBOR modules
- At least one external user/contributor opens an issue or PR
- At least one faculty member reviews the codebase and confirms technical soundness

Dependencies

- **Python:** 3.10+
- **Core libraries:** NumPy, Pandas, SciPy, scikit-learn
- **ML/DL:** PyTorch (for neural nets and RL), Stable-Baselines3 (for RL algorithms)
- **Data:** WRDS/CRSP access (or Yahoo Finance fallback for development)
- **Compute:** Standard laptop for most operations; GPU optional for DL training

Roadmap

Version	Target Date	Deliverables
v0.1.0	Mar 1, 2026	Core structure, data loaders, basic backtest engine
v0.2.0	Mar 31, 2026	HRP, Monte Carlo VaR, risk decomposition
v0.3.0	May 15, 2026	Neural volatility forecaster trained and validated
v0.4.0	Jun 30, 2026	Deep RL agents operational, ABF Q1/Q2 integrated
v0.5.0	Sep 30, 2026	Production-ready release, external validation

References

[1] Lopez de Prado, M. (2016). Building diversified portfolios that outperform out of sample. *Journal of Portfolio Management*, 42(4), 59-69.

[2] Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654-669.

[3] Bucci, A. (2020). Realized volatility forecasting with neural networks. *Journal of Financial Econometrics*, 18(3), 502-531.

[4] Deng, Y., et al. (2017). Deep direct reinforcement learning for financial signal representation and trading. *IEEE Transactions on Neural Networks and Learning Systems*, 28(3), 653-664.

[5] Théate, T., & Ernst, D. (2021). An application of deep reinforcement learning to algorithmic trading. *Expert Systems with Applications*, 173, 114632.

HARBOR: Risk-Aware Portfolio Framework — Technical PRD v1.0

Project: HARBOR (Hierarchical Autonomous Risk-Based Optimization & Research)
Type: Open-Source Python Framework
Version: 1.0.0
Date: February 18, 2026
Owner: Lewis Smith

Executive Summary

HARBOR is a research-grade, open-source portfolio optimization and risk management framework built for depth over breadth. It implements state-of-the-art mathematical models (Hierarchical Risk Parity, Monte Carlo simulation, risk decomposition) and modern

ML/RL/DL techniques in a clean, modular architecture designed for autonomous operation and scientific reproducibility.

Core philosophy: Few components, deep implementation, rigorous math, production-quality code.

Objectives

- Provide a mathematically rigorous portfolio construction and risk management toolkit that institutional researchers can audit, extend, and trust
- Implement advanced techniques (HRP, Monte Carlo VaR/CVaR, neural volatility forecasting, deep RL agents) with full mathematical transparency and reproducibility
- Serve as the computational foundation for Artificial Behavioral Finance (ABF) research—enabling experiments on systematic strategy behavior, crowding, and correlation regimes
- Maintain production-grade engineering standards (tests, CI, versioning, docs) while prioritizing research flexibility

Non-Goals

- Live trading or execution infrastructure (research/backtest only)
- Supporting every asset class or exotic derivative (focus: equities, with clean extensibility)
- Building a GUI or web dashboard (command-line and notebook interfaces only)
- Competing with commercial platforms (this is open research infrastructure)

Technical Architecture

Core Modules

```
harbor/
data/
loaders.py # SP500, CRSP, survivorship-bias-free loaders
universe.py # Historical constituent membership
risk/
hierarchical.py # Hierarchical Risk Parity (HRP)
```

monte_carlo.py # Monte Carlo VaR/CVaR simulation
decomposition.py # Risk attribution and marginal risk
correlation.py # Dynamic correlation estimation, regime detection
portfolio/
optimization.py # Mean-variance, risk-parity, constrained optimization
rebalancing.py # Turnover-aware rebalancing logic
signals/
momentum.py # Time-series and cross-sectional momentum
volatility.py # Realized vol, GARCH, vol-targeting signals
crowding.py # Signal similarity and crowding proxies
ml/
volatility/
nn_forecaster.py # LSTM/GRU neural volatility models
training.py # Training loops, validation, hyperparameter search
behavior_agents/
rl_agent.py # Deep RL portfolio agents (PPO-style)
envs.py # Gym-compatible portfolio environments
rewards.py # Behavioral reward functions (loss aversion, etc.)
backtest/
engine.py # Event-driven backtester
metrics.py # Sharpe, Sortino, max DD, Calmar, VaR metrics
attribution.py # Performance attribution by signal/factor
abf/
q1_persistence.py # ABF Question 1 pipeline
q2_crowding.py # ABF Question 2 pipeline

Design Principles

1. **Mathematical transparency:** Every risk model includes docstring with closed-form equations and citations
2. **Reproducibility:** Fixed random seeds, versioned configs, deterministic pipelines
3. **Modularity:** Each component is independently testable and replaceable
4. **Performance:** Vectorized operations (NumPy/Pandas), optional GPU for DL (PyTorch)

Deep Technical Components

1. Hierarchical Risk Parity (HRP)

Mathematical foundation:

HRP addresses instability in mean-variance optimization by using hierarchical clustering and recursive bisection to allocate risk[1].

Algorithm:

1. Compute correlation matrix C from returns
2. Use hierarchical clustering (e.g., single-linkage) on distance metric $d_{ij} = \sqrt{(1 - \rho_{ij})/2}$
3. Perform recursive bisection to allocate risk inversely proportional to cluster variance

Implementation requirements:

- Covariance estimation: sample covariance, shrinkage estimators (Ledoit-Wolf), or exponentially weighted
- Distance metrics: Euclidean on correlations, or alternative linkages (Ward, complete)
- Validation: compare against naive risk parity and mean-variance

Output: Weight vector $w \in \mathbb{R}^N$ satisfying $\sum w_i = 1, w_i \geq 0$, optimized for diversification without relying on expected returns.

2. Monte Carlo Risk Simulation

Purpose: Estimate tail risk (VaR, CVaR/ES) and stress-test portfolios under simulated scenarios.

Methods:

- **Historical simulation:** Bootstrap from historical return distribution
- **Parametric simulation:** Assume multivariate normal or Student-t, draw N scenarios
- **GARCH-based simulation:** Forecast volatility paths, simulate returns conditional on vol dynamics

Metrics computed:

$$\mathrm{VaR}_\alpha = -\inf\{x : P(L \le x) \ge \alpha\}$$

$$\mathrm{CVaR}_\alpha = \mathbb{E}[L \mid L \ge \mathrm{VaR}_\alpha]$$

where L is portfolio loss, α is confidence level (e.g., 0.95, 0.99).

Implementation:

- 10,000+ simulated paths per risk calculation
- Antithetic variates for variance reduction
- Parallel simulation for large universes (multiprocessing or GPU)

3. Neural Volatility Forecasting

Motivation: Improve volatility estimates for risk models and vol-targeting strategies using recurrent neural networks[2][3].

Architecture:

- **Input:** Rolling window of returns, realized volatility, volume (e.g., 60-day lookback)
- **Model:** LSTM or GRU with 2–3 hidden layers, dropout for regularization
- **Output:** Forecasted volatility $\hat{\sigma}_{t+h}$ for horizons $h \in \{1, 5, 21\}$ days

Training:

- Loss: MSE on volatility forecast error, or quantile loss for distributional forecasts
- Validation: rolling-window cross-validation to avoid lookahead bias
- Benchmarks: compare against historical vol, GARCH(1,1), realized vol

Risk integration:

Use $\hat{\sigma}_{t+1}$ from neural forecaster in:

- Volatility-targeting rules: scale position size inversely with $\hat{\sigma}$

- Covariance matrix estimation: replace diagonal elements with neural forecasts

4. Deep RL Behavioral Agents

Purpose: Model portfolio agents with behavioral biases (loss aversion, overconfidence) using deep reinforcement learning[4][5].

Environment:

- State: portfolio weights, returns, volatility, crowding proxies
- Actions: weight adjustments (continuous or discretized)
- Reward: modified Sharpe ratio with behavioral penalties (e.g., extra penalty for drawdowns to encode loss aversion)

Agent architecture:

- Actor: neural network mapping state s_t to action a_t (policy $\pi_\theta(a|s)$)
- Critic: neural network estimating value function $V_\phi(s)$ or $Q_\phi(s, a)$
- Algorithm: PPO (Proximal Policy Optimization) or SAC (Soft Actor-Critic)

Behavioral modifications:

- **Loss aversion:** Asymmetric reward: $r_t = \begin{cases} \alpha \cdot \text{gain} & \text{if gain} \\ \beta \cdot \text{loss} & \text{if loss} \end{cases}$, $\beta > \alpha$
- **Overconfidence:** Underweight volatility in state representation or reward
- **Disposition effect:** Bias toward holding losers, selling winners (encoded in action penalties)

Research application:

Train multiple agents with similar features but different behavioral parameters, then simulate their collective behavior to study crowding and correlation amplification (ABF Question 2).

5. Risk Decomposition and Attribution

Marginal risk contribution:

For asset i in portfolio with weights w, covariance Σ:

$$\mathrm{MRC}_i = \frac{\partial \sigma_p}{\partial w_i} = \frac{(\Sigma w)_i}{\sigma_p}$$

Risk contribution:

$$\mathrm{RC}_i = w_i \cdot \mathrm{MRC}_i$$

Validation: $\sum_{i=1}^{N} \mathrm{RC}_i = \sigma_p$ (portfolio volatility).

Use cases:

- Identify concentrated risk exposures
- Rebalance toward equal risk contribution (risk parity)
- Detect when single assets dominate portfolio risk

ML/RL Technical Depth

Volatility Forecasting — Detailed Spec

Data preprocessing:

- Returns: log returns $r_t = \log(P_t/P_{t-1})$
- Realized volatility: $\mathrm{RV}_t = \sqrt{\sum_{i=1}^{n} r_{t,i}^2}$ (intraday if available, else close-to-close)
- Normalization: z-score standardization per asset

Model:

```
LSTM(
input_size=3, # [return, realized_vol, volume]
hidden_size=64,
num_layers=2,
dropout=0.2
)
Linear(64, 1) # Output: volatility forecast
```

Training procedure:

1. Split: 70% train, 15% validation, 15% test (chronological, no shuffling)
2. Optimizer: Adam, learning rate 0.001 with ReduceLROnPlateau
3. Early stopping: patience=10 epochs on validation loss
4. Batch size: 32, epochs: 100 max

Evaluation metrics:

- Statistical: MSE, MAE on volatility forecast
- Risk: VaR violation rate (proportion of days where realized loss exceeds predicted VaR)
- Economic: Sharpe ratio of vol-targeted portfolio using NN forecasts vs GARCH

Deep RL Agent — Detailed Spec

Environment (Gym-compatible):

class PortfolioEnv:
observation_space: Box(shape=(N_assets * 3,)) # [weights, returns, vols]
action_space: Box(shape=(N_assets,), low=-0.1, high=0.1) # weight changes

```
def step(action):
    # Apply action, compute new weights, calculate reward
    # Deduct transaction costs proportional to turnover
    return next_state, reward, done, info
```

Agent (PPO):

Actor: MLP(state_dim, [256, 128], action_dim) with tanh activation
Critic: MLP(state_dim, [256, 128], 1) with linear output
Policy: Gaussian with learned std
Training: PPO clip=0.2, GAE lambda=0.95, 2048 steps per update

Behavioral reward (loss aversion example):

def reward(returns, weights, prev_weights):
pnl = np.dot(returns, weights)
turnover_cost = transaction_cost * np.sum(np.abs(weights - prev_weights))

```
if pnl >= 0:
    behavioral_pnl = pnl  # Normal gain
else:
    behavioral_pnl = 2.5 * pnl  # Amplify loss (loss aversion)

return behavioral_pnl - turnover_cost
```

Training:

- Train for 500k steps over historical SP500 data (daily rebalancing)
- Save checkpoints every 50k steps
- Evaluate on held-out test period (2024–2026)

Integration with ABF Research

HARBOR provides the computational substrate for ABF experiments:

ABF Component	HARBOR Module
Vol-targeting strategies	signals/volatility.py, portfolio/optimization.py
Shock detection	risk/correlation.py, abf/q1_persistence.py
Crowding proxies	signals/crowding.py, abf/q2_crowding.py
Agent simulations	ml/behavior_agents/
Backtest validation	backtest/engine.py, backtest/metrics.py

Research findings from ABF (e.g., "reduce exposure when crowding proxy > threshold") are implemented as regime-aware portfolio construction rules and validated via backtest comparison.

Success Metrics

Engineering Quality

- **Test coverage:** $\geq 80\%$ for core modules (risk, portfolio, backtest)
- **Documentation:** Every public function has docstring with equations and references
- **CI/CD:** GitHub Actions running tests on every PR
- **Reproducibility:** All experiments runnable with single `make reproduce` command

Technical Depth

- HRP implementation validated against published benchmarks (e.g., Lopez de Prado papers)
- Monte Carlo VaR backtested against realized losses (violation rate within $\pm 2\%$ of nominal)
- Neural volatility forecaster beats GARCH baseline by $\geq 10\%$ on MSE and VaR violation rate
- Deep RL agent achieves Sharpe ratio ≥ 1.0 on test period with realistic transaction costs

Research Enablement

- ABF Q1 and Q2 experiments run end-to-end using HARBOR modules
- At least one external user/contributor opens an issue or PR
- At least one faculty member reviews the codebase and confirms technical soundness

Dependencies

- **Python:** 3.10+
- **Core libraries:** NumPy, Pandas, SciPy, scikit-learn
- **ML/DL:** PyTorch (for neural nets and RL), Stable-Baselines3 (for RL algorithms)
- **Data:** WRDS/CRSP access (or Yahoo Finance fallback for development)

- **Compute:** Standard laptop for most operations; GPU optional for DL training

Roadmap

Version	Target Date	Deliverables
v0.1.0	Mar 1, 2026	Core structure, data loaders, basic backtest engine
v0.2.0	Mar 31, 2026	HRP, Monte Carlo VaR, risk decomposition
v0.3.0	May 15, 2026	Neural volatility forecaster trained and validated
v0.4.0	Jun 30, 2026	Deep RL agents operational, ABF Q1/Q2 integrated
v0.5.0	Sep 30, 2026	Production-ready release, external validation

References

[1] Lopez de Prado, M. (2016). Building diversified portfolios that outperform out of sample. *Journal of Portfolio Management*, 42(4), 59-69.

[2] Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654-669.

[3] Bucci, A. (2020). Realized volatility forecasting with neural networks. *Journal of Financial Econometrics*, 18(3), 502-531.

[4] Deng, Y., et al. (2017). Deep direct reinforcement learning for financial signal representation and trading. *IEEE Transactions on Neural Networks and Learning Systems*, 28(3), 653-664.

[5] Théate, T., & Ernst, D. (2021). An application of deep reinforcement learning to algorithmic trading. *Expert Systems with Applications*, 173, 114632.